

HelixSkills

HelixSkills

CLI AI 에이전트를 위한 헌법 기반 관리형 스킬 시스템

요약

HelixSkills는 CLI AI 에이전트를 위한 스킬 시스템으로, Helix Constitution를 하위 모듈로 상속하여 모든 보편적 거버넌스 규칙이 무조건 적용됩니다. 이 시스템은 설치 가능한 에이전트 스킬, MCP 도구 서버, Claude 코드 플러그인, 그리고 재사용 가능한 엔진을 등록 가능하고 문서화된 카탈로그로 제공합니다.

간략 설명

HelixSkills는 CLI AI 에이전트를 위한 스킬 시스템입니다. Helix Constitution를 하위 모듈로 포함하여 모든 보편적 규칙을 적용한 후, 등록 가능한 스킬(액션 접두어, 미디어 검증, 멀티트랙, 세션 동기화, 작업 가능 항목 라이프사이클 등), 두 개의 MCP 도구 서버, 두 개의 Claude 코드 플러그인, 그리고 재사용 가능한 엔진을 제공합니다.

상세 설명

HelixSkills(skills 저장소, Apache-2.0 라이선스)는 CLI AI 에이전트를 위한 스킬 시스템으로, 의도적인 역발상으로 시작합니다. 즉, 기능보다 거버넌스가 우선입니다. 이 시스템은 Helix Constitution를 constitution/ 하위 모듈로 상속하여, constitution/CLAUDE.md와 constitution/Constitution.md의 모든 보편적 규칙이 관습적으로 준수되는 것이 아니라 프로젝트 트리에 물리적으로 통합된 규칙으로 무조건 적용됩니다. HelixSkills를 채택한 에이전트는 헌법적 규칙을 회피할 수 없습니다. 규칙은 코드와 함께 이동합니다.

대부분의 “스킬 프레임워크”가 추상화에 의존하는 것과 달리, HelixSkills는 구체적이고 등록 가능한 인벤토리를 제공합니다. register.sh를 통해 설치되는 일곱 가지 헌법 스킬은 다음과 같습니다: action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle — 이 스킬들은 중

간에서 고급 수준에 이르기까지, 체계적인 액션 네이밍부터 미디어 검증, 작업 단위의 전체 라이프사이클에 이르기까지 모든 것을 아우릅니다. 추가로 초안 단계의 스킬(Android 개요, Java/Kotlin 언어, Linux OS)도 이미 인덱싱되어 활성화 대기 중입니다. 두 개의 MCP 도구 서버(media-validator, scheduled-work)는 이러한 스킬을 Model Context Protocol를 통해 에이전트에 제공하며, 두 개의 Claude 코드 플러그인(helix, scheduled-work)은 동일한 기능을 에이전트 런타임에 직접 통합합니다. 하나의 스킬 세트가 에이전트가 사용하는 모든 인터페이스를 통해 접근 가능합니다.

카탈로그 하단에는 네 가지 깊이 1의 재사용 가능한 엔진이 있습니다 — continuum(구현 완료), session_orchestrator, token_optimizer, clickup_sync(설계 중) — 이는 스킬이 동일한 기반을 재구축하지 않도록 하는 공유 인프라입니다. token_optimizer만 하더라도 vasic-digital 생태계 패키지(TOON, Embeddings, VectorDB, Normalize, conversation)와 HelixDevelopment의 LLMProvider에 이르는 명시적 의존성 그래프를 선언하여, 크로스 레포 연결이 암묵적이 아닌 감사가 가능하도록 설계되었습니다. 전체 시스템은 체계적인 문서로 관리됩니다: 스킬 카탈로그, 자동 생성된 스킬 그래프 인덱스, 각 레포별 상세 페이지, 그리고 아직 해결되지 않은 사항을 명시한 ‘Gaps & Risks’ 등록부까지 포함됩니다. 또한 시스템은 GitHub, GitLab, GitFlic, GitVerse에 걸쳐 미러링되어 복원력과 지역별 접근성을 보장합니다.

콘텐츠

왜 만들었는가

CLI와 AI 에이전트에게는 일관되고, 통제되며, 재사용 가능한 기능이 필요합니다. 각기 다른 규칙을 재창조하는 임시 스크립트가 아니라요. HelixSkills는 에이전트에게 공유된 규범에 기반한 패키지화되고 등록 가능한 스킬 세트를 제공하기 위해 만들어졌습니다. 이를 통해 채택한 모든 에이전트와 프로젝트에서 행동의 일관성과 감사 가능성을 유지할 수 있습니다.

혁신적인 이유

에이전트 기능이 규범을 준수하는 것은 설계 단계에서부터 보장되며, 개인의 노력에 의존하지 않습니다. 모든 스킬은 규범 하위 모듈에 의해 뒷받침되는 통제되고, 버전이 관리되며, 설치 가능한 단위입니다. 에이전트가 스킬을 등록하는 순간, 해당 스킬은 표준 규칙 세트를 상속받아 일탈의 여지가 사라집니다. 이는 이전에는 실현 불가능했던 것을 가능하게 합니다. 즉, 한 에이전트나 프로젝트에서 다른 곳으로 기능을 이전하더라도, 이미 동일한 거버넌스에 묶여 도착한다는 것을 알 수 있습니다. 이 기능은 각기 다른 규칙을 재창조하는 임시 접착 스크립트가 아니라, 표준 인터페이스(MCP 서버와 Claude 코드 플러그인)를 통해 제공됩니다.

혁신적인 점

- 하위 모듈로서의 **Constitution**: 보편적인 거버넌스 규칙은 복사되지 않고 상속됩니다. 트리에 마운트되어 모든 소비 에이전트가 동일한 표준 규칙 세트에 구속되며, 업데이트는 하나의 진실 공급원에서만 이루어집니다. 수십 개의 오래된 복사본이 아니라고요.
- 자기 등록 단위로 제공되는 스킬(register.sh): 설치 시 자동으로 연결되며, 자동 생성된 스킬 그래프 인덱스에 통합되어 카탈로그가 항상 실제 설치된 내용과 동기화된 상태로 유지됩니다.
- 다중 인터페이스 노출: 동일한 스킬 세트가 MCP 도구 서버와 Claude 코드 플러그인을 통해 에이전트에 제공됩니다. 한 번 작성하면 에이전트가 사용하는 어떤 런타임과도 소통할 수 있습니다.
- 생태계 전반에서 공유되는 재사용 가능한 **depth-1** 엔진(continuum, token_optimizer, session_orchestrator, clickup_sync): 각 엔진은 암묵적인 결합이 아닌 명시적이고 감사 가능한 교차 저장소 의존성 선언을 포함합니다.

가장 큰 기술적 도전 과제와 해결 방법

- 다양한 스킬과 에이전트에서 일관되고 규칙을 준수하는 행동 유지 — 스킬마다 거버넌스를 재구현하면 시간이 지남에 따라 발산이 보장됩니다. 이를 해결하기 위해 Helix Constitution를 하위 모듈로 마운트하여 constitution/CLAUDE.md와 constitution/Constitution.md의 규칙이 무조건 적용되고, 하나의 상위 소스에서만 업데이트되도록 했습니다. 복사되어 방치되는 것이 아니라고요.
- 증가하는 스킬 세트를 설치 가능하고 발견 가능하게 만들기 — 카탈로그가 있어도 아무도 그 안에 있는 것을 찾거나 설치할 수 없다면 무용지물입니다. 이를 해결하기 위해 설치 시 각 스킬을 연결하는 register.sh 등록 방식을 도입했으며, 자동 생성된 INDEX 스킬 그래프와 저장소별 상세 문서를 통해 발견 기능이 실제 상태를 자동으로 반영하도록 했습니다.
- 다른 런타임을 사용하는 에이전트에 도달하기 — 동일한 기능을 호스트마다 다시 구축해서는 안 됩니다. 이를 해결하기 위해 하나의 스킬 세트를 MCP 도구 서버 정의(constitution/mcp/ 하위)와 Claude 코드 플러그인(constitution/plugins/ 하위)에 모두 패키징하여 단일 구현을 다양한 인터페이스로 노출했습니다.

콘텐츠

기술 스택

- **Shell(주 언어)** — 에이전트가 존재하는 모든 환경에서 런타임을 먼저 부트스트랩하지 않고도 설치 및 등록 도구가 실행되어야 하므로 선택됨. register.sh와 install_upstreams를 구동하며, 진입 장벽을 의존성 없이 이식 가능하게 유지함.
- **Git 서브모듈** — 거버넌스를 중복 없이 상속하기 위해 선택됨. Helix Constitution는 constitution/에 라이브 참조로 마운트되어 규칙 업데이트가 한 포인터를 통해 전파되며, 복사-붙여넣기 후 잊히는 일이 없음.

- **Model Context Protocol(MCP)** — 에이전트를 위한 표준 런타임 종립 도구 인터페이스로 선택됨. constitution/mcp/ 하위에 두 개의 MCP 서버(media-validator, scheduled-work)가 정의되어 스킬을 호출 가능한 도구로 노출함.
- **Claude** 코드 플러그인 — 에이전트 런타임에 스킬을 네이티브로 통합하며 추가 접착 코드 없이 동작하도록 선택됨. 두 개의 플러그인(helix, scheduled-work)은 constitution/plugins/에 제공되며, 다른 호스트를 위한 MCP 인터페이스를 미러링함.
- 재사용 가능한 엔진(**continuum, token_optimizer, session_orchestrator, clickup_sync**) — 개별 스킬에서 공통 로직을 분리하여 프로젝트 간 재사용을 위해 선택됨. 예를 들어 token_optimizer는 vasic-digital 패키지(TOON, Embeddings, VectorDB, Normalize, conversation)와 HelixDevelopment의 LLMProvider에 선언된 의존성을 통해 연결되며, 코드 중복이 없음.
- 멀티 호스트 **Git** 미러링(**GitHub, GitLab, GitFlic, GitVerse**) — 단일 호스트 장애나 지역 차단이 접근을 차단하지 않도록 선택됨. 동일한 저장소가 네 개의 포지에서 실시간으로 유지되어 복원력과 접근성을 보장함.

현황 및 솔직 노트

- **현황:** 베타. 일곱 개의 헌법 스킬, 두 개의 MCP 서버, 두 개의 플러그인이 배포됨. 초안 스킬은 인덱싱되어 활성화 대기 중이며, 네 개의 depth-1 엔진 중 세 개(session_orchestrator, token_optimizer, clickup_sync)는 아직 설계 단계임.
- README에서는 프로젝트를 helix_skills로 언급하지만, 공식 GitHub 경로는 HelixDevelopment/skills임. README의 발견 건수는 자체 보고된 수치임.

우선 순위: Helix-주.